

تقرير تنفيذ واختبار الطلبين الثاني و الخامس

محمد ملاذ احمد

ملاحظة : هناك صور اختبارات كثيرة في المجلد غير مضمنة بالتقرير

إنما التقرير لقياس الفرق قبل و بعد

المحور	الغرض	الأدوات
الطلب الثاني	ضبط استهلاك الموارد تحت الضغط ومنع انهيار التطبيق عند تراكم الطلبات	TypeORM ,Redis ,BullMQ ,NestJS
الطلب الخامس	توزيع الطلبات على أكثر من نسخة من التطبيق بدل الاعتماد على نسخة واحدة	Docker Compose ,Nginx ,least_conn ,خوارزمية
اختبار الأداء	قياس زمن الاستجابة ونسبة الأخطاء ومعدل الطلبات تحت الضغط	Aggregate تقارير , Apache JMeter Summary و

المكون	الدور في النظام
NestJS	تنظيم التطبيق إلى وحدات، وتوفير الحراس، والحقن الاعتمادي، و ربط المتحكمات بالخدمات
TypeORM مع PostgreSQL	تخزين البيانات وتنفيذ المعاملات وقفل السجلات عند الحاجة لمنع التعارض
BullMQ و Redis	تخزين المهام المؤجلة وتشغيلها خارج دورة الطلب المباشر
Nginx	استقبال الطلبات وتوزيعها على نسخ التطبيق المختلفة
Docker Compose	تشغيل قاعدة البيانات و Redis ونسخ التطبيق و Nginx ضمن بيئة واحدة
JMeter	تنفيذ اختبارات الضغط وقراءة المؤشرات الإحصائية للأداء

الحلول المستخدمة في الطلب الثاني

الحل	مكان التطبيق	السبب
Rate Limiting	src/app.module.ts	تحديد عدد الطلبات المسموح بها خلال فترة زمنية، وإرجاع الحالة 429 عند تجاوز الحد
Queue Backpressure	src/carts/carts.controller.ts	فحص عدد المهام المنتظرة قبل قبول مهمة جديدة في السلة، وإرجاع الحالة 503 عند امتلاء الصف
Worker Concurrency	src/carts/carts.processor.ts	تحديد عدد المهام التي يمكن للمعالج تنفيذها بالتوازي حتى لا يضغط على قاعدة البيانات و Redis
DB Connection Pool	src/app.module.ts	تحديد عدد اتصالات قاعدة البيانات بدل تركها غير مضبوطة أثناء الضغط
Redis Memory Limit	docker-compose.yml	وضع حد لذاكرة Redis مع سياسة عدم حذف المهام تلقائياً، حتى لا تضيق المهام عند الضغط

أفضل طلب لاختبار هذا الجزء هو طلب **إضافة منتج إلى السلة**، لأنه يضيف مهمة إلى صف السلة ويؤثر على قاعدة البيانات عند معالجة المهمة.

```
POST /carts/add
-> ThrottlerGuard
-> ensureCartQueueCapacity()
-> BullMQ cart queue
-> CartConsumer
-> CartsService
-> PostgreSQL transaction
```

الطلب الخامس: توزيع الحمل

تم تنفيذ الطلب الخامس بتشغيل تلت `instances` من التطبيق و وضع موزع طلبات أمامها لتمرير كل طلب إلى النسخة المناسبة. تم تعريف النسخ داخل ملف إعداد الحاويات، ثم تم تمرير الطلبات إليها من خلال إعدادات موزع الطلبات . و استخدامنا خوارزمية `least conn` تختار هذه الخوارزمية النسخة التي لديها أقل عدد من الاتصالات النشطة حالياً. لأن الطلبات ليست كلها بنفس الزمن؛ بعض الطلبات خفيف وبعضها يتعامل مع قاعدة البيانات . هذا الـ `proxy` يلي هو NGINX

```
upstream nest_backend {
    least_conn;
    server api1:3000;
    server api2:3000;
    server api3:3000;
}
```

لماذا لم نستخدم Round Robin

التوزيع بالدور يرسل الطلبات بالتناوب بغض النظر عن انشغال النسخة الحالية. أما خوارزمية أقل الاتصالات فتراعي عدد الاتصالات النشطة لذلك تعطي فرصة أفضل للنسخة الأقل انشغالاً
في مشروع يحتوي على معاملات وقاعدة بيانات هذا الاختيار أكثر مرونة .

الاختبار

العنصر	القيمة
الطلب المختبر	GET /products
عدد العينات	16000 طلب
نوع التقارير	تقرير تجميعي وتقرير ملخص
هدف القراءة	قياس أثر الضغط العالي على طلب قراءة قبل و بعد الاعتماد على ال load balancing .

القيم من الصور المرفقة مباشرة. تظهر النتائج أن الضغط كان كبيراً على السيناريو المختبر، وأن نسبة الأخطاء مرتفعة ، وهذا يدعم الحاجة إلى حلول الطلب الثاني والخامس .

المؤشر	القيمة	التفسير
عدد العينات	16000	عدد الطلبات التي تم إرسالها إلى طلب قراءة المنتجات
متوسط الزمن	3054ms	متوسط الاستجابة يقارب 3.05 ثانية، وهو مرتفع لطلب قراءة
الوسيط	3674ms	نصف الطلبات تقريباً احتاجت إلى 3.67 ثانية أو أقل
خط 90%	5371ms	تسعون بالمئة من الطلبات كانت أقل من 5.37 ثانية
خط 95%	5522ms	خمسة وتسعون بالمئة من الطلبات كانت أقل من 5.52 ثانية
خط 99%	5951ms	تسعة وتسعون بالمئة من الطلبات كانت أقل من 5.95 ثانية
أعلى زمن	6446ms	أطول استجابة وصلت إلى نحو 6.44 ثانية
نسبة الأخطاء	58.51%	نسبة أخطاء عالية جداً وتدل أن الضغط تجاوز قدرة السيناريو المختبر
معدل التنفيذ	114.1/sec	النظام عالج أو سجل نحو 114 طلباً في الثانية لكن مع نسبة أخطاء عالية

Aggregate Repor

Name:Aggregate Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:☐ Errors☐ Successes

Configure

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/...	Sent KB/sec
Get Products	16000	3054	3674	5371	5522	5951	0	6446	%٥٨,٥١	١١١٤,١/sec	١٢٧٧,٢١	٢٨٨,١٩
TOTAL	16000	3054	3674	5371	5522	5951	0	6446	%٥٨,٥١	١١١٤,١/sec	١٢٧٧,٢١	٢٨٨,١٩

المؤشر	القيمة	التفسير
عدد العينات	20000	عدد الطلبات التي تم إرسالها إلى طلب قراءة المنتجات
متوسط الزمن	410ms	زمن أقل
أعلى زمن	6446ms	نفس أعلى زمن ظاهر في التقرير السابق
نسبة الأخطاء	%0	لم يفشل شيء
معدل التنفيذ	sec/172.4	ارتفع قليلا , معدل الطلبات في الثانية، ويجب قراءته مع نسبة الأخطاء لا وحده

Summary Report

Name:Summary Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:☐ Errors☐ Successes

Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Get Products	20000	410	1	2634	492.18	0.00%	478.5/sec	327.58	172.43	701.0
TOTAL	20000	410	1	2634	492.18	0.00%	478.5/sec	327.58	172.43	701.0

مقارنة عامة بين ما قبل التحسين وما بعده

النقطة	قبل التحسين	بعد التحسين
قبول الطلبات	قد يقبل النظام عدداً كبيراً من الطلبات حتى تظهر أخطاء اتصال أو ضغط على قاعدة البيانات	يتم رفض الضغط الزائد بشكل واضح عبر 429 أو 503 بدلاً من الانهيار
عمليات السلة	الطلبات قد تتراكم دون فحص واضح لسعة الصف	يتم فحص سعة صف السلة قبل إدخال مهمة جديدة
قاعدة البيانات	احتمال ضغط أعلى بسبب عدم ضبط الاتصالات بالشكل المطلوب	يوجد حد للاتصالات قاعدة البيانات لتقليل الاختناق
توزيع الطلبات	الضغط قد يتركز على نسخة واحدة أو توزيع غير واضح	توجد ثلاث نسخ خلف Nginx ويتم توزيع الطلبات بخوارزمية least_conn
قراءة نتائج الاختبار	الأخطاء العالية قد تظهر ك 502 أو انقطاع اتصال عند تجاوز القدرة	الأخطاء المتوقعة تصبح مضبوطة أكثر مثل 429 أو 503 عند تجاوز الحدود
استهلاك الموارد	-	زاد استهلاك الـ cpu +80% إلى +200% وقت الضغط زاد استهلاك الـ ram +200% تقريباً عند تشغيل الثلاث كونتainers